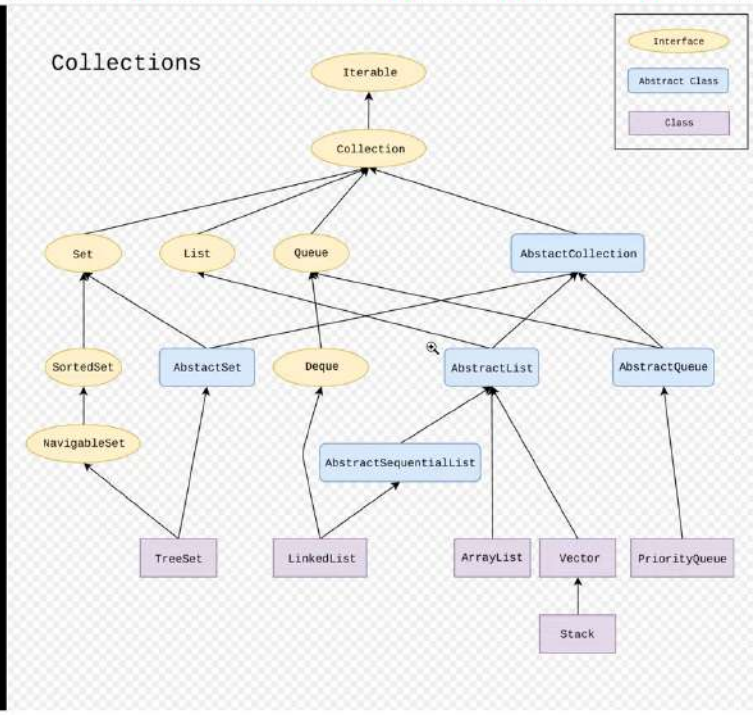
Collections Hierarchy in Java



java.util.Collection class and interface hierarchy

More details

Reminun · Own work

CC BY-SA 4.0

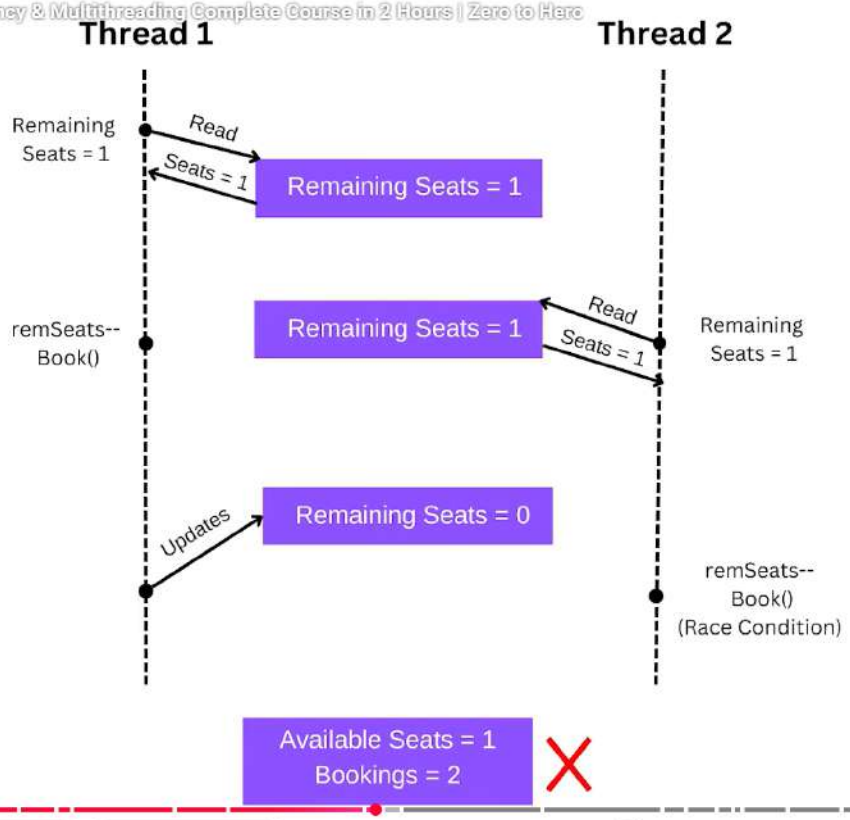
6:02 / 9:44

In this video

Synchronization :

- Threads share the same memory space, i.e. they can share resources (objects)
- However, there are critical situations where it is desirable that only one thread at a time has access to a shared resource.




Thread 1
Thread 2
Remaining Seats = 1
Read
Seats = 1
Remaining Seats = 1
remSeats--
Book()
Remaining Seats = 1
Read
Seats = 1
Remaining Seats = 1
Updates
Remaining Seats = 0
remSeats--
Book()
(Race Condition)
Available Seats = 1
Bookings = 2



Synchronized Methods :

- While a thread is inside a synchronized method of an object, all other threads that wish to execute this synchronized method or any other synchronized method of the object will have to wait.
- This restriction does not apply to the thread that already has the lock and is executing a synchronized method of the object.
- Such a method can invoke other synchronized methods of the object without being blocked.
- The non-synchronized methods of the object can always be called at any time by any thread.



Rules of Synchronization :

- A thread must acquire the object lock associated with a shared resource before it can enter the shared resource.
- The runtime system ensures that no other thread can enter a shared resource if another thread already holds the object lock associated with it.
- If a thread cannot immediately acquire the object lock, it is blocked, i.e., it must wait for the lock to become available.
- When a thread exits a shared resource, the runtime system ensures that the object lock is also relinquished. If another thread is waiting for this object lock, it can try to acquire the lock in order to gain access to the shared resource.



Rules of Synchronization :

- It should be made clear that programs should not make any assumptions about the order in which threads are granted ownership of a lock.











Static Synchronized Methods :

- A thread acquiring the lock of a class to execute a static synchronized method has no effect on any thread acquiring the lock on any object of the class to execute a synchronized instance method.



Static Synchronized Methods :

- A thread acquiring the lock of a class to execute a static synchronized method has no effect on any thread acquiring the lock on any object of the class to execute a synchronized instance method.
- In other words, synchronization of static methods in a class is independent from the synchronization of instance methods on objects of the class.
- A subclass decides whether the new definition of an inherited synchronized method will remain synchronized in the subclass.



Race Condition :

It occurs when two or more threads simultaneously update the same value(stackTopIndex) and, as a consequence, leave the value in an undefined or inconsistent state.



Synchronized Blocks :

- Whereas execution of synchronized methods of an object is synchronized on the lock of the object, the synchronized block allows execution of arbitrary code to be synchronized on the lock of an arbitrary object.
- The general form of the synchronized statement is as follows: synchronized (object ref expression) { <code block> }
- The object ref expression must evaluate to a non-null reference value, otherwise a NullPointerException is thrown.



Summary :

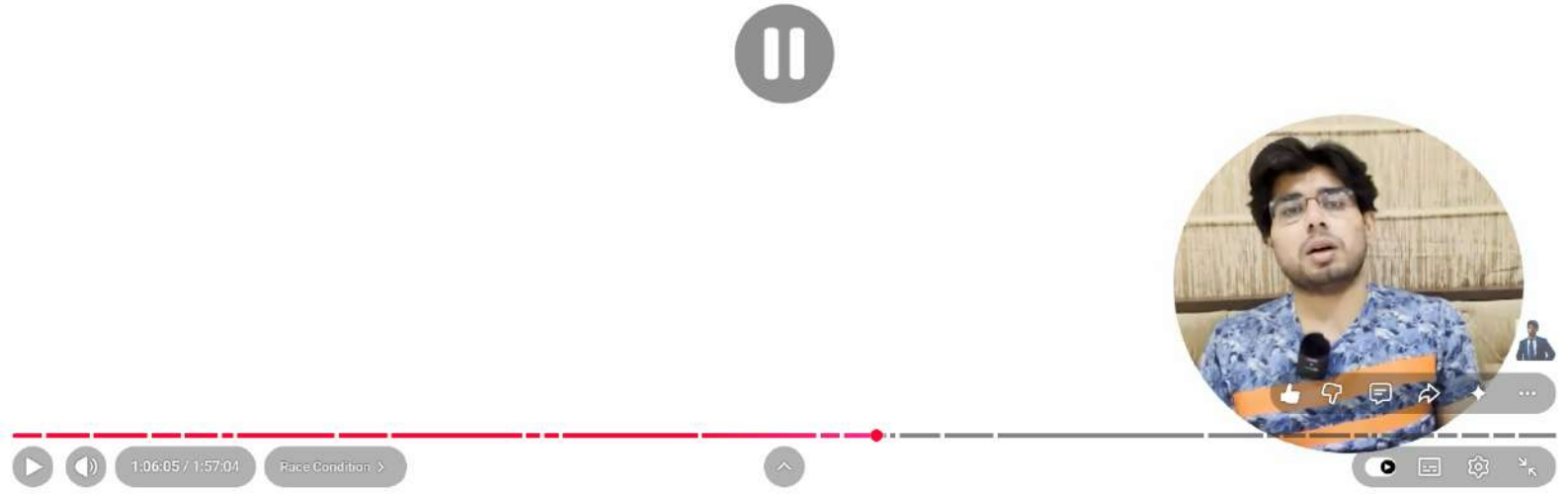
A thread can hold a lock on an object :

- By executing a synchronized instance method of the object. (this)
- By executing the body of a synchronized block that synchronizes on the object. (this)
- By executing a synchronized static method of a class or a block inside a static method (in which case, the object is the Class object representing the class in the JVM)

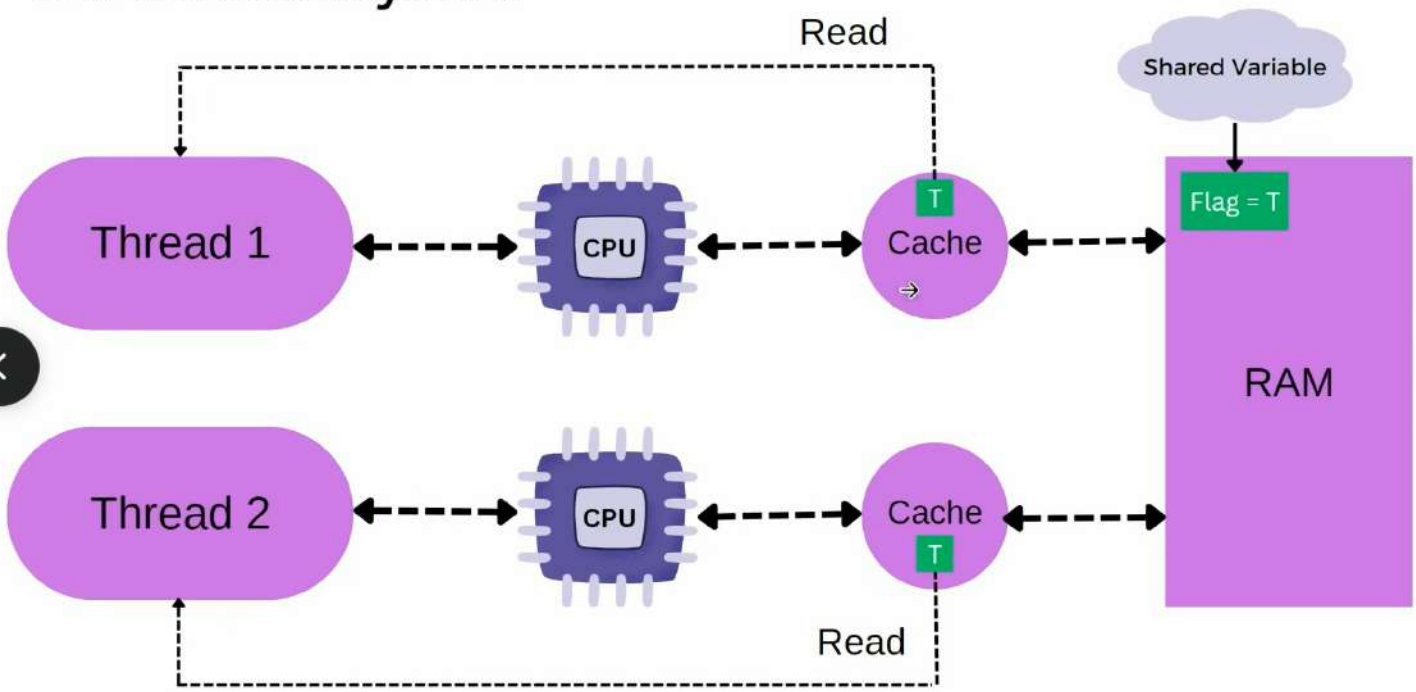


Thread safety :

It's the term used to describe the design of classes that ensure that the state of their objects is always consistent, even when the objects are used concurrently by multiple threads. Eg StringBuffer.

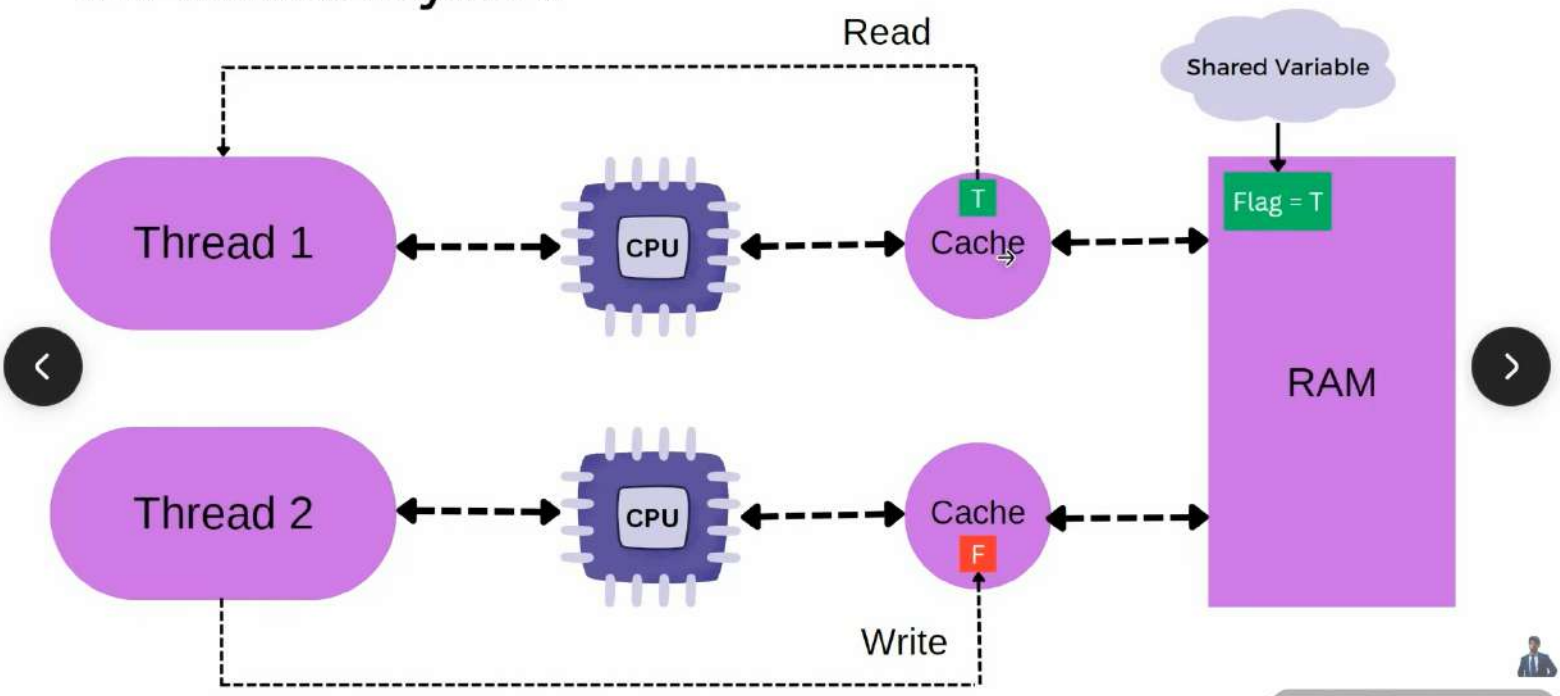


The Volatile Keyword



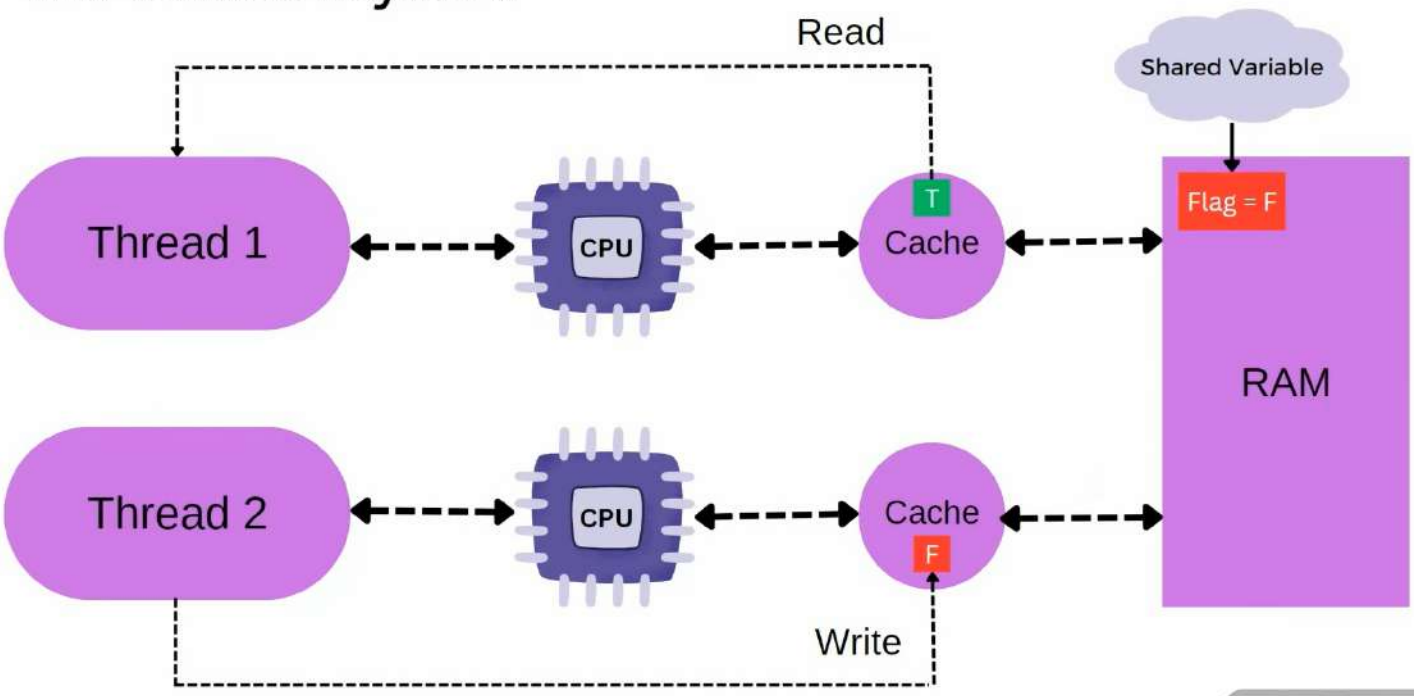
Java Concurrency & Multithreading Complete Course in 2 Hours | Zero to Hero

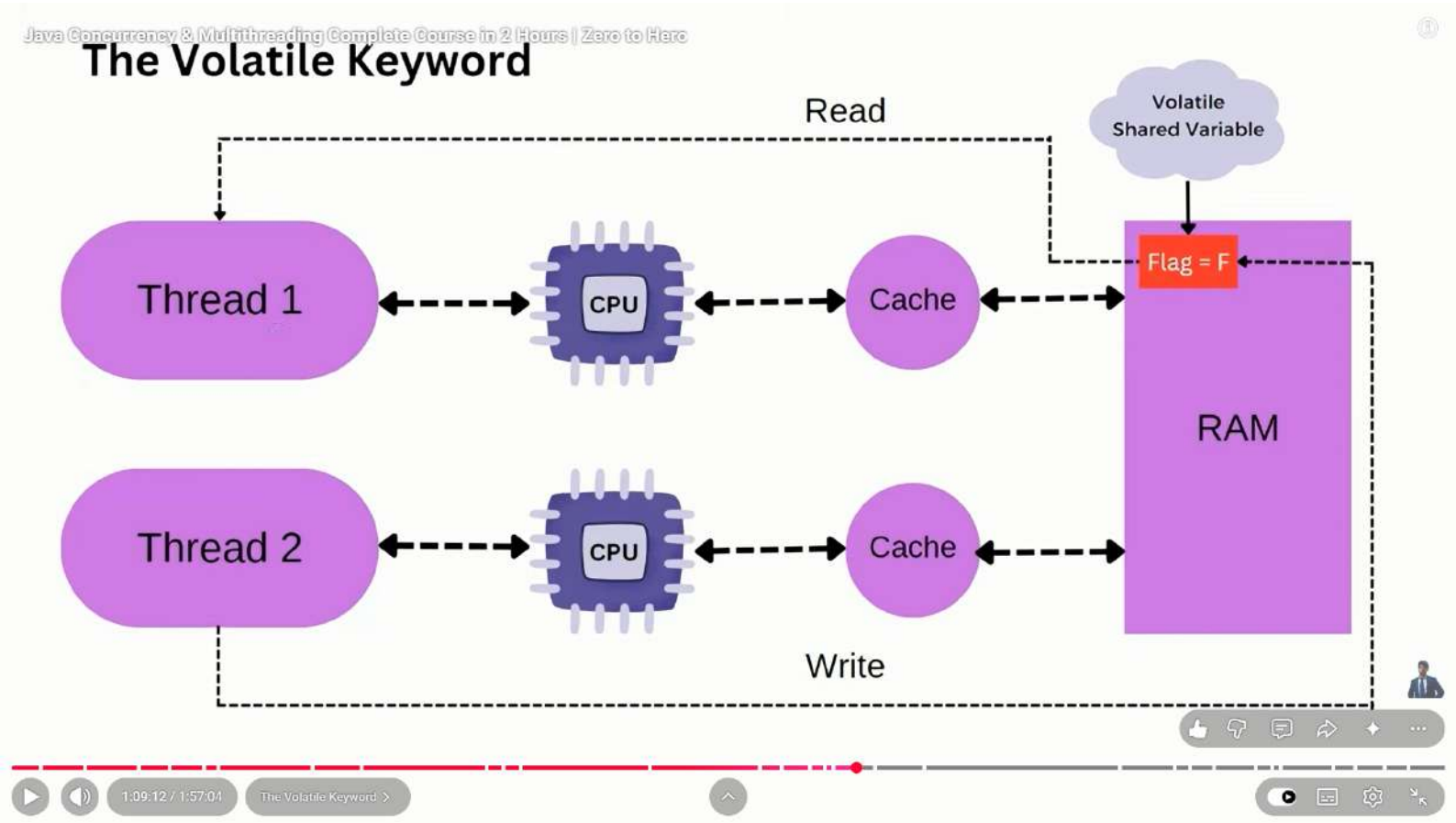
The Volatile Keyword

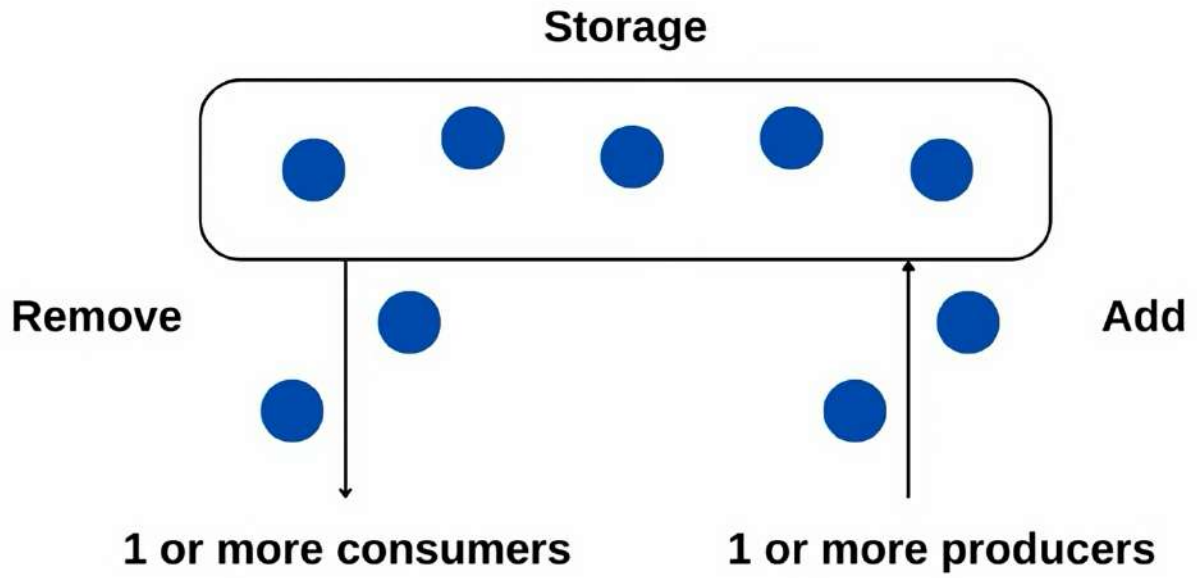


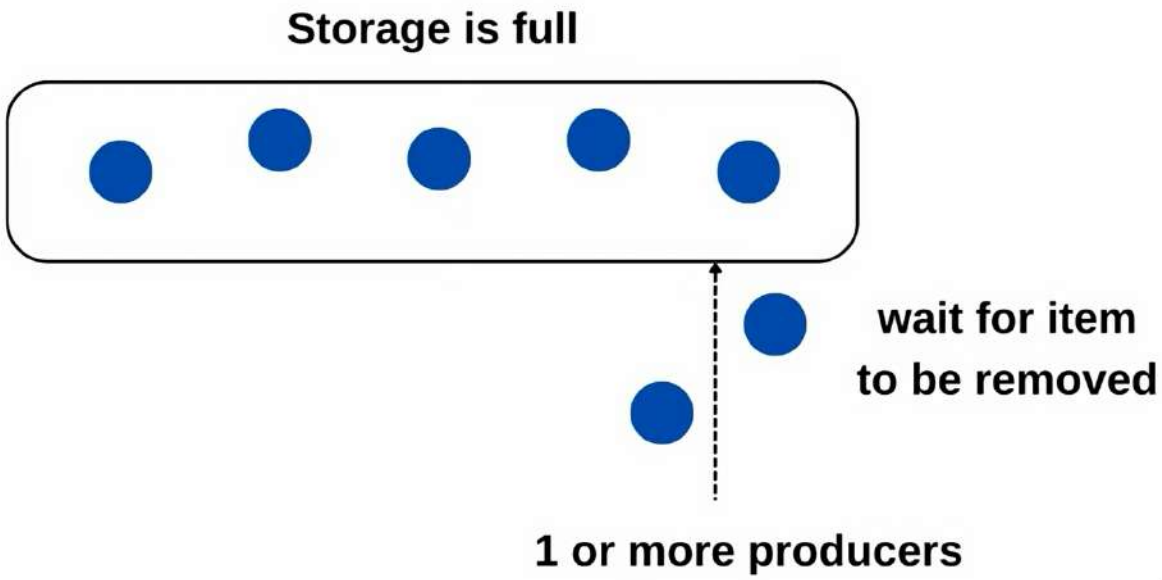
Java Concurrency & Multithreading Complete Course in 2 Hours | Zero to Hero

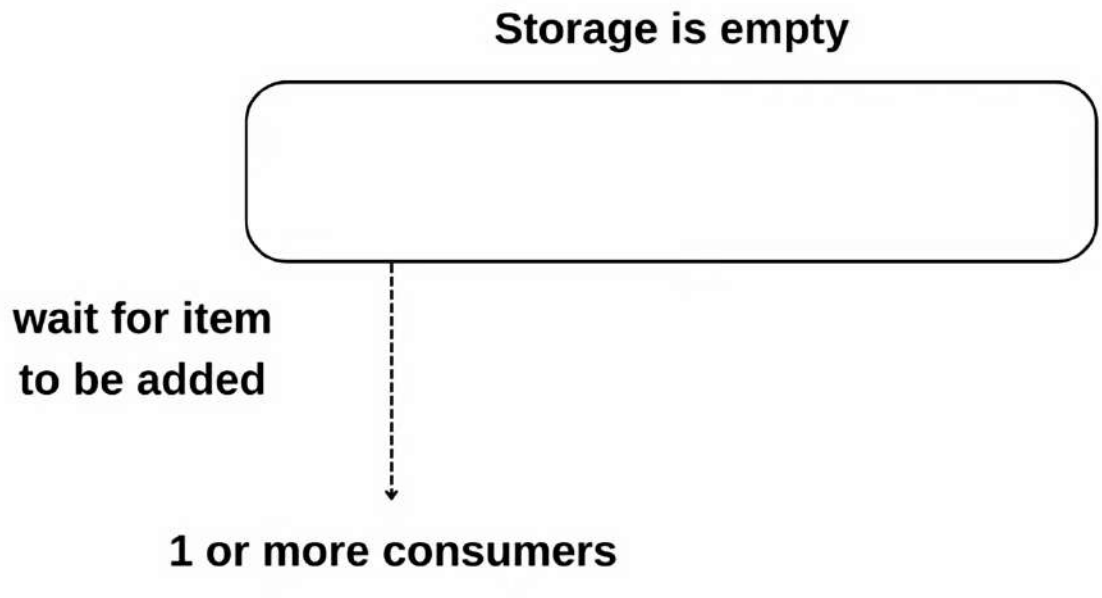
The Volatile Keyword




Java Concurrency & Multithreading Complete Course in 2 Hours | Zero to Hero
The Volatile Keyword
Read
Volatile
Shared Variable
Thread 1
CPU
Cache
Flag = F
RAM
Thread 2
CPU
Cache
Write
1:09:12 / 1:57:04
The Volatile Keyword

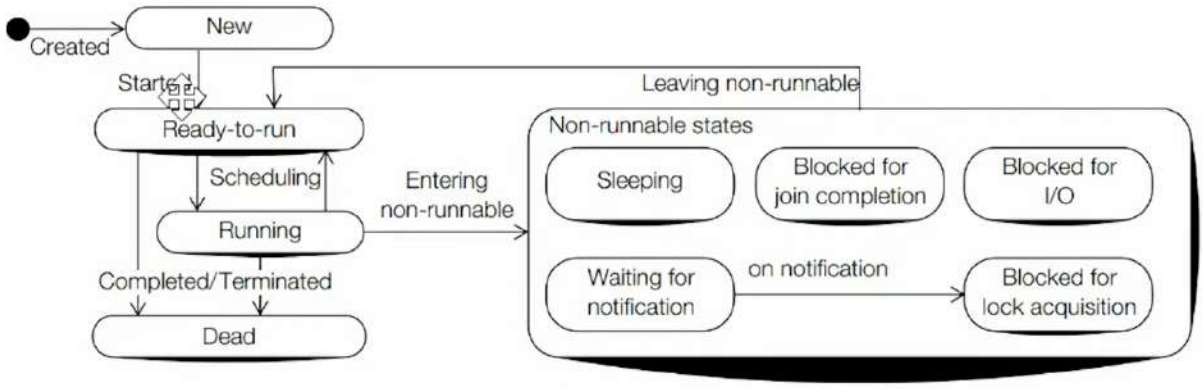

Storage
Remove
Add
1 or more consumers
1 or more producers


Storage is full
wait for item
to be removed
1 or more producers


Storage is empty
wait for item
to be added
1 or more consumers

Thread Transitions

Thread States

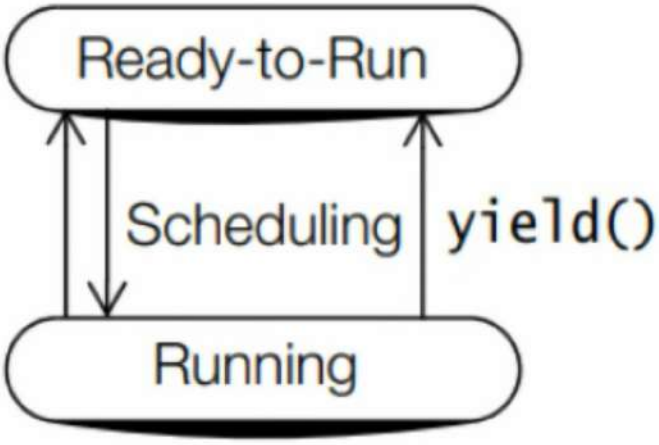


Thread States

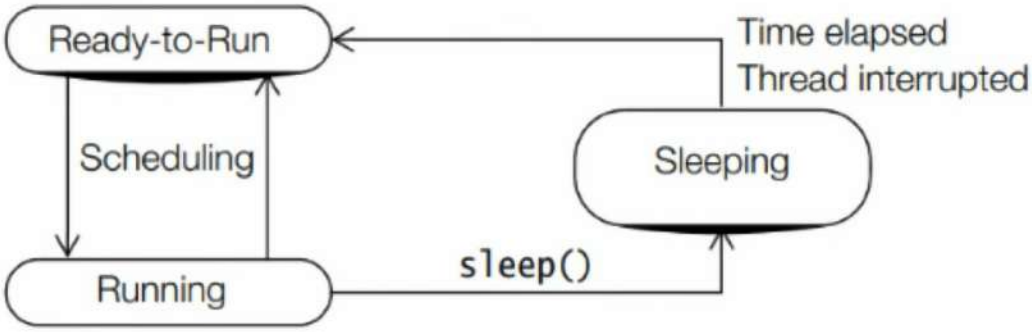
Constant in the `Thread.State` enum type	**State in Figure 13.3**	**Description of the thread**
`NEW`	*New*	Created but not yet started.
`RUNNABLE`	*Runnable*	Executing in the JVM.
`BLOCKED`	*Blocked for lock acquisition*	Blocked while waiting for a lock.
`WAITING`	*Waiting for notify,* *Blocked for join completion*	Waiting indefinitely for another thread to perform a particular action.
`TIMED_WAITING`	*Sleeping,* *Waiting for notify,* *Blocked for join completion*	Waiting for another thread to perform an action for up to a specified time.
`TERMINATED`	*Dead*	Completed execution.



Running and Yielding :

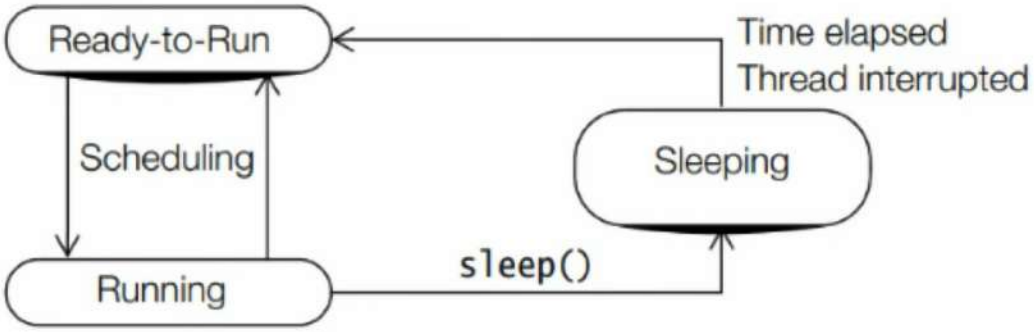


Sleeping and Waking Up :

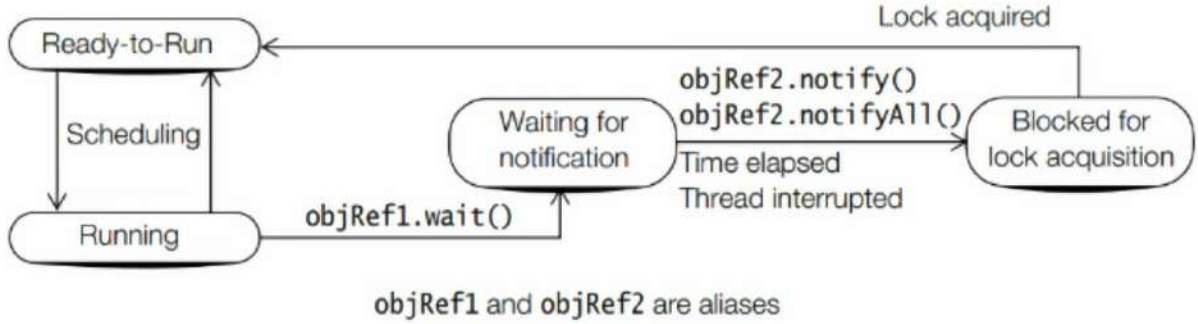




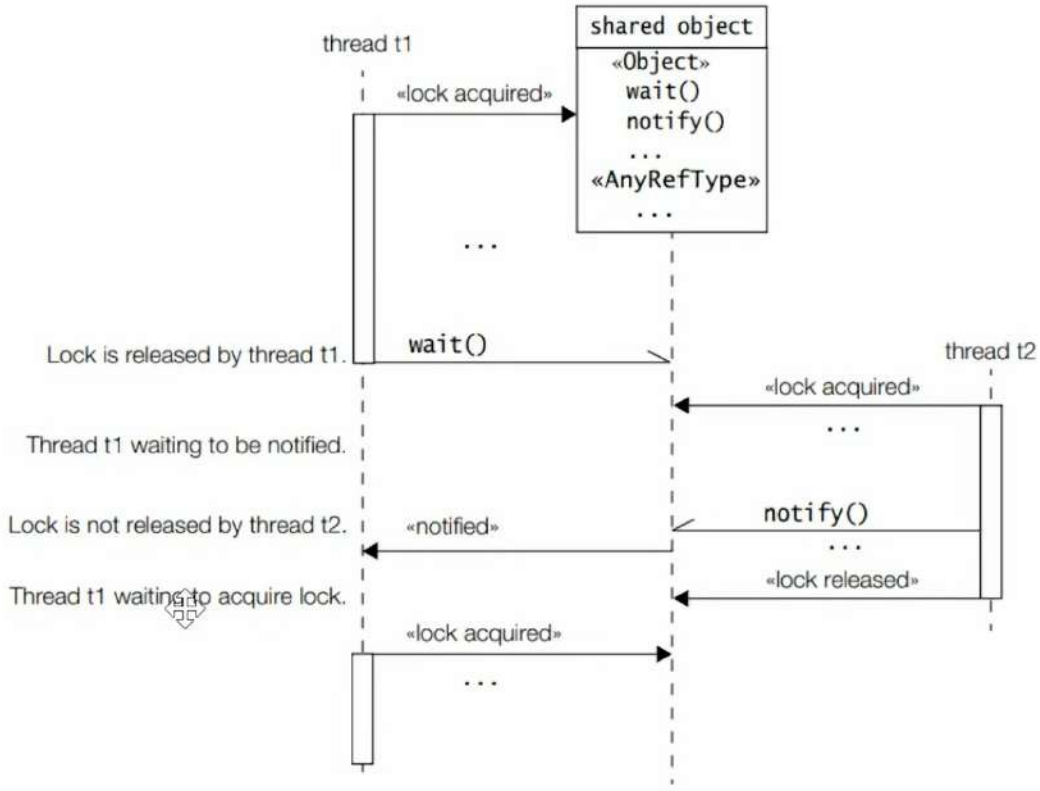
Sleeping and Waking Up :



Waiting and Notifying :



objRef1 and objRef2 are aliases


thread t1
shared object
«Object»
wait()
notify()
...
«AnyRefType»
...
«lock acquired»
...
Lock is released by thread t1.
wait()
thread t2
«lock acquired»
...
Thread t1 waiting to be notified.
Lock is not released by thread t2.
«notified»
notify()
...
«lock released»
Thread t1 waiting to acquire lock.
«lock acquired»
...

Waiting and Notifying :

A thread in the Waiting-for-notification state can be awakened by the occurrence of any one of these three incidents:

- Another thread invokes the notify() method on the object of the waiting thread, and the waiting thread is selected as the thread to be awakened.
- The waiting thread times out.
- Another thread interrupts the waiting thread.



Notify :

- Invoking the notify() method on an object wakes up a single thread that is waiting for the lock of this object.
- The selection of a thread to awaken is dependent on the thread policies implemented by the JVM.
- On being notified, a waiting thread first transits to the Blocked-for-lock-acquisition state to acquire the lock on the object, and not directly to the Ready-to-run state.
- The thread is also removed from the wait set of the object.



Important Methods :

- final void wait(long timeout) throws InterruptedException
- final void wait(long timeout, int nanos) throws InterruptedException
- final void wait() throws InterruptedException
- final void notify()
- final void notifyAll()



Timed Out :

- The wait() call specified the time the thread should wait before being timed out, if it was not awakened by being notified.
- The awakened thread competes in the usual manner to execute again.
- The awakened thread has no way of knowing whether it was timed out or woken up by one of the notification methods.

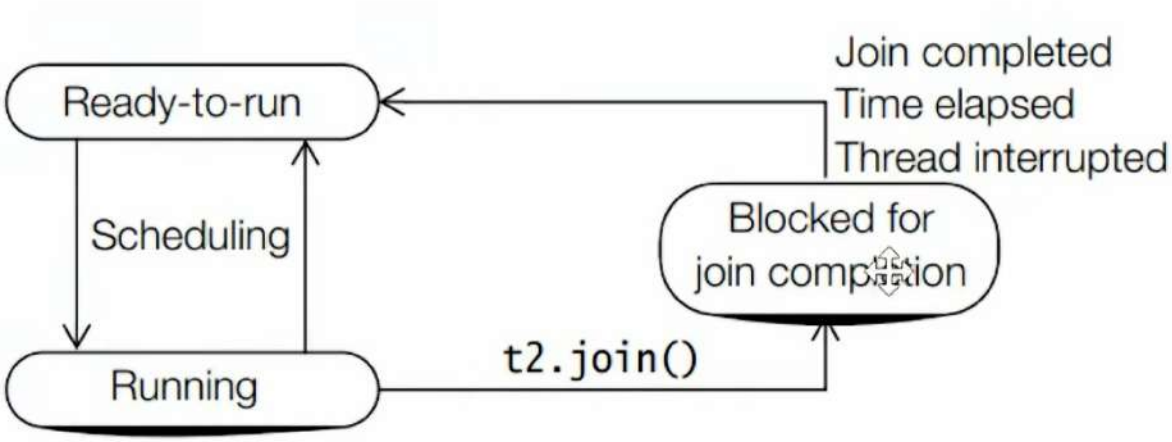


Interrupted :

- Another thread invoked the interrupt() method on the waiting thread.
- The awakened thread is enabled , but the return from the wait() call will result in an InterruptedException if and when the awakened thread finally gets a chance to run.
- The code invoking the wait() method must be prepared to handle this checked exception.



Thread Joining



Thread Priorities :

- Threads are assigned priorities that the thread scheduler can use to determine how the threads will be scheduled.







Thread Priorities :

- Threads are assigned priorities that the thread scheduler can use to determine how the threads will be scheduled.
- The thread scheduler can use thread priorities to determine which thread gets to run.



Thread Priorities :

- Priorities are integer values from 1 (lowest priority given by the constant Thread. MIN_PRIORITY) to 10 (highest priority given by the constant Thread.MAX_PRIORITY). The default priority is 5 (Thread.NORM_PRIORITY).



Thread Scheduler :

- Schedulers in JVM implementations usually employ one of the two following strategies
  1. Preemptive scheduling.
  2. Time-Sliced or Round-Robin scheduling.



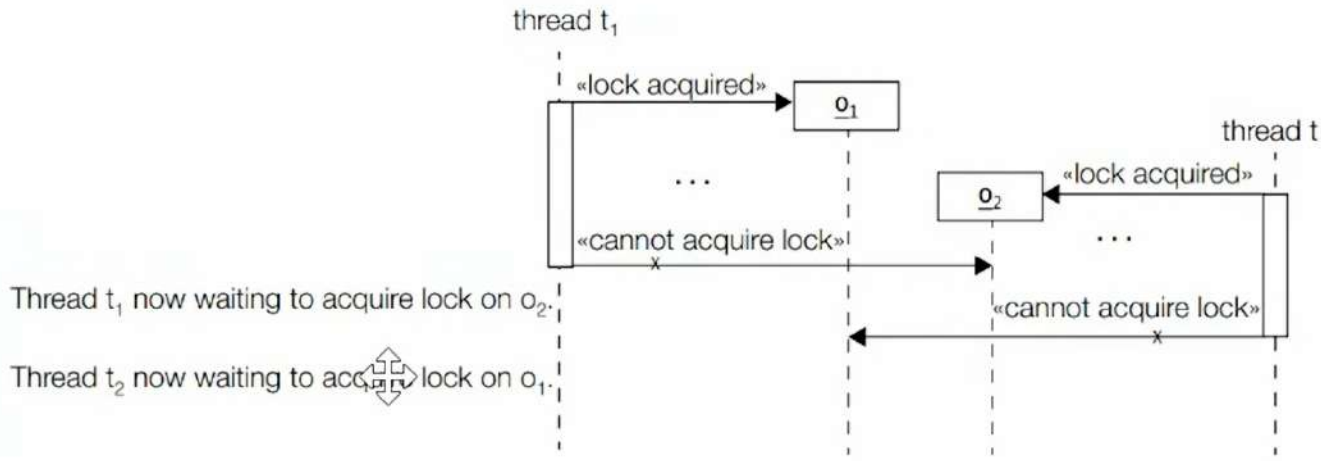


Deadlocks :

- A deadlock is a situation where a thread is waiting for an object lock that another thread holds, and this second thread is waiting for an object lock that the first thread holds.
- Since each thread is waiting for the other thread to relinquish a lock, they both remain waiting forever in the Blocked-for-lock-acquisition state.
- The threads are said to be deadlocked.



DEADLOCKS


thread t_1
«lock acquired»
o_1
thread t
«lock acquired»
o_2
«cannot acquire lock»
«cannot acquire lock»
Thread t_1 now waiting to acquire lock on o_2.
Thread t_2 now waiting to acquire lock on o_1.